

# LAN Party

## Descrierea proiectului

Liga studenților organizează un eveniment (LAN Party) în care studenții se întâlnesc pentru a juca în rețea un anumit joc. Concursul este structurat ca un turneu, iar liga își dorește o soluție software care să automatizeze generarea meciurilor și a etapelor din turneu.

Proiectul presupune înțelegerea modului de funcționare a structurilor de date (liste, stive, cozi, arbori și grafuri) și operarea cu aceste structuri.

Pentru a facilita dezvoltarea programului, proiectul este organizat în trei etape (task-uri), cu termene de predare diferite.

Funcționarea soluției voastre va fi verificată automat, utilizând un program (*checkerul*) și mai multe date de intrare, salvate în fișiere separate. Concret, pentru fiecare etapă există trei teste diferite.

Programul vostru trebuie să citească aceste fișiere de intrare și să genereze, pentru fiecare, un fișier de ieșire, cu rezultatele. Atât numele fișierelor de intrare, cât și a celor de ieșire vor fi transmise în programul vostru ca argumente.

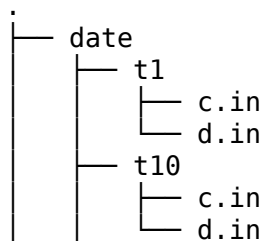
## Obțineți datele și checkerul

Downloadați tot conținutul de aici.

**Nu modificați structura directorului!**

## Datele de intrare

Directorul [date](#) conține câte un subdirector pentru fiecare astfel de test.



Un test este definit de 2 fișiere de intrare, pe care programul trebuie să le primească sub formă de argumente

- **c.in**, conține codificarea numărului cerinței
- **d.in**, conține datele de intrare propriu-zise <sup>1</sup>, necesare rezolvării.

*De ce există două fișiere de intrare pentru fiecare test?*

Fiecare task vizează altă etapă din dezvoltarea soluției, cu accent pe o anumită structură de date. Din acest motiv, conținutul și forma fișierelor de intrare diferă de la task la task.

Soluția pentru etapa 1, de exemplu, trebuie verificată doar pentru fișierele de intrare corespunzătoare acestora (testul 1, 2 și 3). Fișierul **c.in** conține numărul etapei asociate fișierului cu datele propriu-zise, **d.in**.

Programul vostru trebuie să primească, sub formă de argumente, numele ambelor fișiere **c.in** și **d.in**. În funcție de conținutul **c.in**, trebuie să efectueze operațiile specifice respectivului task.

## Setup

*Ce este checkerul?*

Checkerul este un program care automatizează procesul de verificare a corectitudinii soluției voastre. Un astfel de program poate fi scris în orice limbaj de programare. Cel pe care îl veți folosi voi este scris sub forma unui *script bash*.

*De ce este nevoie de automatizarea procesului de verificare?*

După cum am menționat anterior, soluția voastră va fi verificată pe date de intrare diferite (teste). Acesta este un scenariu tipic în testarea software, deoarece programele trebuie să funcționeze corect în condiții diferite. Aceste condiții diferite sunt simulate cu date de intrare diferite. În plus, pentru fiecare test, se urmărește:

- corectitudinea soluției (rezultatul în sine să fie corect)
- respectarea formatului în care sunt returnate rezultatele (de ex: valorile să fie separate prin spațiu, nu prin linie nouă etc)

În lipsa unei soluții automate, verificarea ar presupune rularea de mână a programului pentru fiecare test în parte și evaluarea rezultatului.

*Cum funcționează checkerul?*

Checkerul parcurge fiecare director de test și rulează programul vostru pe fișierele de intrare din acel test. De asemenea, creează un director, **out**, în care salvează fișierul de ieșire pe care îl generează programul vostru. Apoi verifică dacă fișierul de ieșire generat de programul vostru este corect.

*Cum știe checkerul care sunt rezultatele corecte?*

Checkerul are acces la un director, **rezultate**, în care se găsește soluția corectă pentru fiecare test. Și voi aveți acces la acest director; folosiți-l pentru a vedea cum ar trebui formateate fișierele voastre de ieșire.

---

<sup>1</sup>Fișierele de intrare conțin denumirile reale ale echipelor participante într-un astfel de eveniment organizat acum câțiva ani. Nu ne asumăm responsabilitatea pentru calitatea umorului participanților.

Checkerul compară, caracter cu caracter, conținutul unui fișier-referință (din directorul **rezultate**) cu cel al fișierului de ieșire generat de programul vostru (din directorul **out**).

Puteți face și voi, manual, această comparație pentru o pereche de fișiere, utilizând site-ul.

## 1 Creați programul vostru

Creați programul vostru și **compilați-l**. Checkerul se așteaptă ca executabilul pe care trebuie să-l ruleze să se numească **lanParty**.

În Figura 1, observați conținutul directorului curent la prima comandă **ls**: directoarele **date** și **rezultate**, fișierul sursă **main.c**, executabilul **lanParty** și checkerul.

```
a@a:/mnt/d/uni/_s2__sd_fsa/2026/tema$ ls
LICENSE Makefile README.md checker.sh date lanParty main.c rezultate
a@a:/mnt/d/uni/_s2__sd_fsa/2026/tema$ ./checker.sh

-----
Executabil = 'lanParty'
-----
[TESTE NORMALE]
Test 1 ..... PASSED (+ 5pt)
Test 2 ..... PASSED (+ 7pt)
Test 3 ..... PASSED (+ 8pt)
Test 4 ..... PASSED (+15pt)
Test 5 ..... PASSED (+15pt)
Test 6 ..... PASSED (+15pt)
Test 7 ..... PASSED (+10pt)
Test 8 ..... PASSED (+15pt)
Test 9 ..... PASSED (+10pt)

TOTAL ..... 100pt

-----
[NOTA FINALA]

NOTA FINALA ..... 100pt

a@a:/mnt/d/uni/_s2__sd_fsa/2026/tema$ ls
LICENSE Makefile README.md checker.sh date lanParty main.c out rezultate
a@a:/mnt/d/uni/_s2__sd_fsa/2026/tema$
```

Figura 1: Checker

## 2 Rulați checkerul

Figura 1 prezintă un exemplu de rulare a checkerului, în care rezultatele sunt corecte pentru toate testele.

Observați conținutul directorului curent după rularea checkerului (a doua comandă **ls**): a fost creat directorul **out**, ce conține fișierele de ieșire generate de program pentru fiecare test.

## 3 Makefile

Cu toate că checkerul a automatizat procesul de verificare (execuția programului pe mai multe teste), alte operații trebuie în continuare făcute manual. De exemplu, compilarea

programului pentru a crea un nou fișier executabil de fiecare dată când faceți o modificare în cod sau ștergerea executabilului pentru a pregăti o nouă compilare.

Astfel de operații se pot automatiza cu ajutorul fișierelor **Makefile**. Un exemplu minimal pentru un astfel de fișier este următorul.

```
build:
    gcc main.c -o lanParty
run:
    ./lanParty ./date/t1/c.in ./date/t1/d.in ./out/out1.out
clean:
    rm lanParty
```

Pentru a folosi regulile scrise în acest Makefile, se execută în linia de comandă:

**make build**, pentru a compila programul

**make run** pentru a-l rula

**Checkerul** utilizează comenzile **build** și **make clean**, deci este necesară existența fișierului Makefile cu aceste comenzi.

### Explicații suplimentare

Această secțiune se adresează celor interesați să afle mai multe despre fișierele Makefile.

Aceste fișiere se pot extinde, în funcție de complexitatea operațiilor necesare.

Spre exemplu, prin adăugarea unei reguli noi **build\_run**, care să execute ambele operații.

Observați, de asemenea, că **run** execută programul pe un singur test. Dacă se dorește rularea programului pe toate testele asociate task-ului 1, se poate modifica regula **run**

```
run:
    ./lanParty ./date/t1/c.in ./date/t1/d.in ./out/out1.out
    ./lanParty ./date/t2/c.in ./date/t2/d.in ./out/out2.out
    ./lanParty ./date/t3/c.in ./date/t3/d.in ./out/out3.out
```

Există și variante mai elegante de a scrie comanda de mai sus, utilizând variabile. Cei interesați pot găsi informații aici.

## Task 1

**Deadline.** 19 Aprilie

**Scop.** Crearea listei de echipe participante

Acest task simulează înscrierea echipelor participante. Fiecare jucător înscris în concurs are asociat un nume, un prenume și un punctaj, ce reprezintă experiența lui. La înscriere, mai mulți participanți aleg să concureze într-o echipă, pentru care aleg un nume. Punctajul echipei la înscriere este media aritmetică a punctelor de experiență corespunzătoare jucătorilor din echipa respectiva.

Un turneu poate găzdui un număr de  $n$  echipe, unde  $n$  este putere a lui 2. În cazul în care în concurs se înscriu un număr de echipe ce nu este putere a lui 2, este necesară o selecție, pentru eliminarea echipelor în plus. Criteriul de selecție este punctajul echipei.

Pentru a reprezenta un participant, se poate folosi o structură ca cea de mai jos. Similar se poate proceda pentru reprezentarea unei echipe.

```
struct Player {
    char* firstName;
    char* secondName;
    int points;
};
```

## Cerințe

1. Creați o listă de elemente de tip Echipă și populați-o cu datele din fișierul `d.in`.
2. Eliminați din lista creată anterior echipele cu cel mai mic punctaj, astfel încât numărul de echipe rămase să fie cea mai apropiată putere a lui 2. În cazul în care mai multe echipe au același scor, eliminați prima echipă găsită în listă. După fiecare eliminare, reluați parcurgerea listei de la început.
3. Scrieți în fișierul de ieșire numele echipelor participante, în ordinea parcurgerii listei de la pasul anterior.

## Testare

Datele de intrare conțin

- numărul de echipe înscrise
- numărul de participanți în echipă, urmat de numele echipei
- numele, prenumele și punctajul fiecărui participant din echipă

**Exemplu input:**

3  
5 Local Area Name  
Paul Ploiesteanu 3  
Mircea Muresanu 2  
Ion Ieseanu 2  
Ovidiu Olteanu 3  
Dumitru Dobrogeanu 3

**4 Unusual Suspects**

Popa Ana 6  
Popescu Maria 10  
Pop Anamaria 6  
Popovici Mariana 2

**5 Lando Nowins**

Sebastian Vesel 2  
Valeriu Bota 6  
Nicolae Lauda 10  
Stefan Ocol 3  
Michi Antoniu 1

**Exemplu output:**

Unusual Suspects  
Lando Nowins

## Task 2

**Deadline.** 10 Mai

**Scop.** Simularea turneului

Echipele rezultate din selecția de la punctul anterior vor intra în turneu, care se desfășoară în runde.

La fiecare rundă, câte două echipe participă într-un meci, ce va fi câștigat de echipa cu punctaj mai mare. În cazul unui meci în care ambele echipe au același punctaj, se va alege prima echipa adăugată în meciul disputat.

În urma victoriei, punctajul fiecărui jucător din echipa câștigătoare crește cu 1.

Echipele câștigătoare la această rundă trec în runda următoare.

## Cerințe

1. Creați o coadă ce va conține meciurile dintre echipe. Echipele vor forma meciuri în ordinea în care se află în lista de la *Task 1*: primele două echipe vor forma primul meci, echipa a treia și echipa a patra vor forma cel de-al doilea meci ș.a.m.d..
2. Creați o stivă în care introduceți echipele câștigătoare și actualizați corespunzător punctajul participanților și al echipei.
3. Extrageți din stivă echipele și adăugați-le din nou coada de meciuri. Prima echipă scoasă din stivă va juca cu a doua echipa scoasă din stivă primul meci.
4. Repetați pașii până la aflarea echipelor de pe primele 8 locuri. Salvați aceste echipe într-o nouă listă. Această listă va fi necesară la *Task 3*. Continuați meciurile până la aflarea echipei câștigătoare.
5. Scrieți în fișierul de ieșire, la fiecare rundă, conținutul cozii de meciuri și cel al stivei de învingători.

## Task 3

**Deadline.** 24 Mai

**Scop.** Afișarea rezultatelor

Clasamentul primelor 8 echipe va fi reprezentat sub forma unui arbore binar de căutare, în care elementele sunt ordonate în funcție de punctajul echipelor. În cazul în care două echipe au același punctaj, ordonarea se va realiza în funcție de nume, descrescător.

### Cerințe

1. Utilizând lista cu primele 8 echipe, construită la *Task 2*, creați arborele binar de căutare.
2. Parcurgeți arborele astfel încât să afișați conținutul în ordine descrescătoare și salvați rezultatul parcurgerii în fișierul de ieșire.